

Function

林致宇

Why we need Function?

- Code re-use
- Readability

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

f(x)

Method Name The Parameter List

$f(x) = x * x + 3 * x + 5$

Method Body

老師的提醒：
初學者不必太在意 Argument 和 Parameter 的差異

參考：<https://notfalse.net/6/arg-vs-param>

$f(5) = 45$

Return Value

Function

```
type name(type para1, type para2, ...) {  
    Body  
}
```

Return Type: the data type of the value returned by the function, or **void** if the function does not return a value.

Method Name

The Parameter List: a comma-delimited list of input parameters, preceded by their data types, enclosed by parentheses, (). If there are no parameters, you must use **empty parentheses**.

int **f**(**int** x) {

int result = x + 1;
return result;

}

Method Body: enclosed between braces

Function

- No return value, no parameter list

```
4 void sayHello() {  
5     cout << "Hello World\n";  
6 }
```

- Multiple Parameters

```
4 void init(char * name, int age, bool isMale) {  
5 }
```

- Multiple return statements

```
4 int f (int x) {  
5     if (x % 2 == 0) return 0;  
6     return 1;  
7 }
```

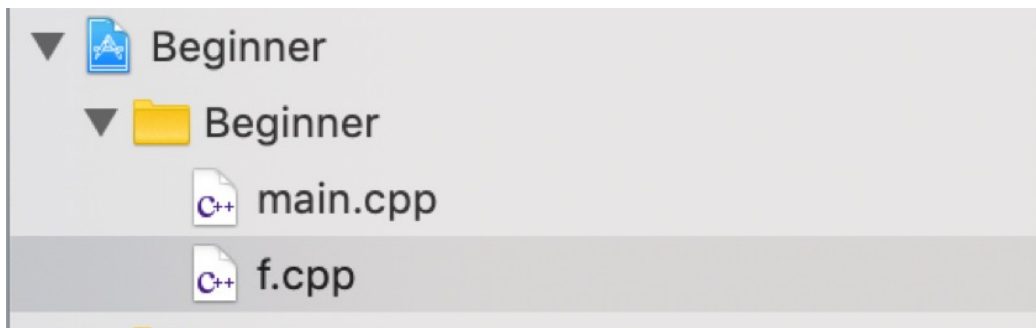
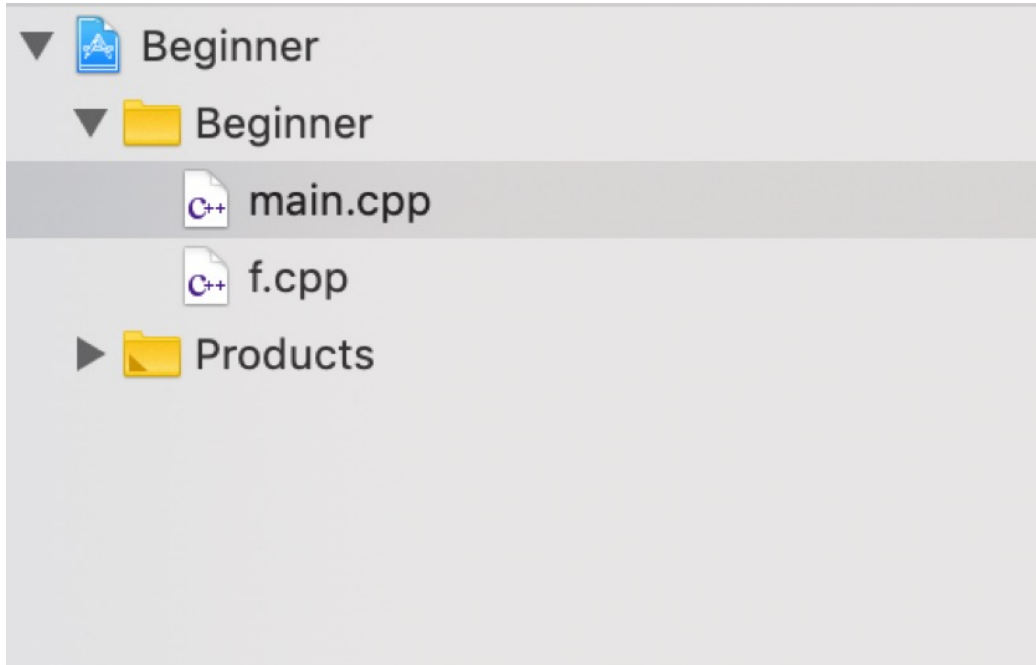
Calling a Function

```
1  #include <iostream>
2  using namespace std;
3
4  int f(int x) {
5      int result = x + 1;
6      return result;
7  }
8
9  int main() {
10
11      cout << f(6) << "\n";
12      cout << f(7) << "\n";
13  }
```


Function Declaration

```
1  #include <iostream>
2  using namespace std;
3
4  int f(int x);
5
6  int main() {
7
8      cout << f(6) << "\n";
9      cout << f(7) << "\n";
10 }
11
12 int f(int x) {
13     int result = x + 1;
14     return result;
15 }
```

Function Declaration



g++ main.cpp f.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int f(int x);
5
6  int main() {
7
8      cout << f(6) << "\n";
9      cout << f(7) << "\n";
10 }
```

```
1  int f(int x) {
2      int result = x + 1;
3      return result;
4  }
5
```

Function Declaration

6_03_f.h

```
1 int f(int x);
```

6_03_main.cpp

```
1 #include<iostream>
2 #include "6_03_f.h"
3 using namespace std;
4
5 int main() {
6
7     cout << f(6) << "\n";
8     cout << f(7) << "\n";
9 }
```

```
g++ 6_03_main.cpp 6_02_f.cpp
```

Default Values

```
1  #include<iostream>
2  #include<string>
3  using namespace std;
4
5  void showPersonalInfo(int id, string name="John", int age = 30) {
6      cout << "ID: " << id << endl;
7      cout << "Name: " << name << endl;
8      cout << "Age: " << age << endl;
9  }
10
11 int main() {
12
13     showPersonalInfo(1);
14     showPersonalInfo(2, "Bob");
15     showPersonalInfo(3, "Peter", 50);
16 }
```

The parameter list of main()

不
考

```
1  #include<iostream>
2  using namespace std;
3
4  int main(int argc, char * argv[]) {
5      |
6      for (int i = 0; i < argc; i++) {
7          |   cout << argv[i] << endl;
8      }
9  }
```

Practice

- (Practice 1) Write a function that accepts two numbers and returns the average of these two numbers.
- (Practice 2) Write a function called findMinimum that accepts an integer array and returns the minimum element.
- (Practice 3) Write a function that can convert Fahrenheit to Celsius.

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

Variable Scope

- Local Variables
- Static Local Variables
- Global Variables
- External Variables
- Shadow

Local Variables

- Variable defined in the function
- main is also a function
- Parameters defined by the function

```
1  #include <iostream>
2  using namespace std;
3
4  int f (int x) {
5      int y = x + 5;
6      return y;
7  }
8
9  int main() {
10     int z = x + y;
11 }
```

2  Use of undeclared identifier 'x'

Static Local Variables

- static variable will only be initialized once

```
4  int f (int x) {  
5      static int y = x;  
6      y++;  
7      return y;  
8  }  
9  
10 int main() {  
11     cout << f(5) << "\n";  
12     cout << f(3) << "\n";  
13 }
```

↗ 只会被初始化一次!

6
7

不会重新初始化
直接执行 Line 6

Global Variables

```
4  int y = 3;
5
6  void f (int x) {
7      y = x;
8  }
9
10 int main() {
11     cout << y << "\n";
12     f(5);
13     cout << y << "\n";
14 }
```

External Variables

6_06_f.cpp

```
1  int y;  
2  
3  void f(int x) {  
4      y = x;  
5  }
```

6_06_f.h

```
1  void f(int x);  
2  extern int y;
```

6_06_main.cpp

```
1  #include<iostream>  
2  #include "6_06_f.h"  
3  using namespace std;  
4  
5  int main() {  
6      cout << y << "\n";  
7      f(5);  
8      cout << y << "\n";  
9  }
```

g++ 6_06_main.cpp 6_06_f.cpp

Shadow

```
4  int x = 100;
5
6  int main() {
7
8      int x = 50;
9
10     cout << x << "\n";
11     cout << ::x << "\n"; // 範圍解析運算子 ::
12 }
```

老師的建議：
一開始命名變數時就盡量避免同名

50

100

Return address

- You should not return the addresses of local variables

```
6  int * larger(int a, int b) {
7      if (a > b) {
8          return &a;
9      } else {
10         return &b;
11     }
12 }
```

```
[lincyu@Chih-YudeMacBook-Pro Chapter6 % g++ 6_20_returnaddr.cpp
6_20_returnaddr.cpp:8:17: warning: address of stack memory associated with parameter 'a'
      returned [-Wreturn-stack-address]
      return &a;
              ^
6_20_returnaddr.cpp:10:17: warning: address of stack memory associated with parameter 'b'
      returned [-Wreturn-stack-address]
      return &b;
              ^
2 warnings generated.
lincyu@Chih-YudeMacBook-Pro Chapter6 %
```

Outline

- Introduction to functions
- Variable Scope
- **Inline functions**
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

思考

Inline Function

- Call Stack

有效率,

但較吃 Mem.

(適合把小的 function 寫成 inline 就好)

- Inline function is a function that is expanded in line when it is called.
- An inline function in C++ is an optimization feature that reduces the **execution time** of a program, just as we have macros in C.

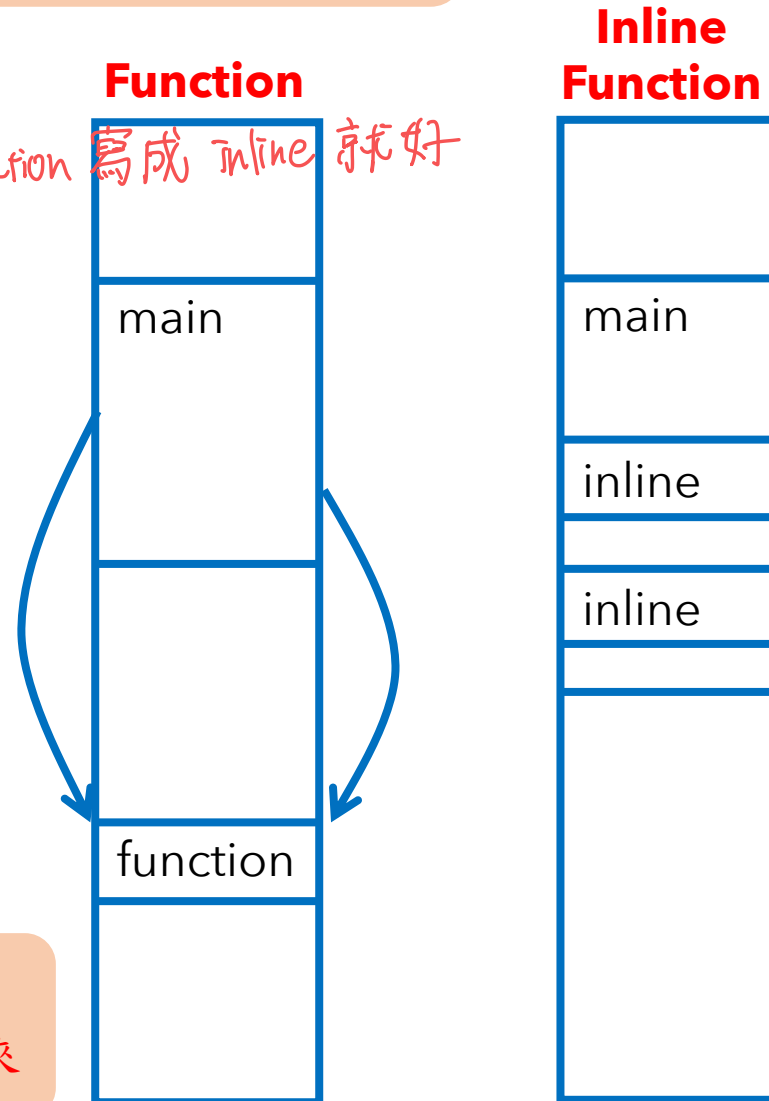
(1) 暫存器狀態要儲存起來

(2) 回去的記憶體位址要儲存起來

老師的提醒：

此議題屬於有點進階議題

inline function 的使用時機？



Inline Function

```
4  inline int sum(int x, int y) {  
5      return (x+y);  
6  }  
7  
8  int main() {  
9  
10     int x = 5, y = 3;  
11  
12     cout << sum(x, y) << "\n";  
13     cout << sum(7, 6) << "\n";  
14 }
```

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

Macro

- 巨集也可像函式一樣用括弧指定參數，但不用指定參數型別

```
4  #define f(x) x * x + 3 * x + 2
5
6  int main() {
7
8      int x = 5, y = 3;
9
10     cout << f(x) << "\n";
11     cout << f(y) << "\n";
12
13 }
```

42

20

Program ended with exit code: 0

Macro

- 前置處理器只是將參數「代換」到巨集中參數出現的位置

老師的建議：
盡量不要使用 Macro，改
成使用 Inline Function

```
4  #define f(x) x * x + 3 * x + 2
5
6  int main() {
7
8      int x = 5;
9
10     cout << f(x+1) << "\n";
11     cout << f(6) << "\n";
12 }
```

29 5 + 1 * 5 + 1 + 3 * 5 + 1 + 2 = 29

56

Program ended with exit code: 0

Inline Function

```
4  inline int f(int x) {  
5      return (x * x + 3 * x + 2);  
6  }  
7  
8  int main() {  
9  
10     int x = 5;  
11  
12     cout << f(x+1) << "\n";  
13     cout << f(6) << "\n";  
14 }
```

```
[lincyu@Chih-YudeMacBook-Pro Programs % g++ 6_11_inline.cpp  
[lincyu@Chih-YudeMacBook-Pro Programs % ./a.out  
56  
56
```

—

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- **Overloading**
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

Overload 会考!

- 使用相同的名稱來定義不同的函式就稱為「多載」(Overload)
- Signature: function name + parameter list
- In C++, two functions can have the same name if the number and/or type of arguments passed is different.
- These functions having the same name, but different arguments are known as overloaded functions.

Overload

```
#include <iostream>
using namespace std;

double avg(int x, int y) {
    return (x+y)/2.0;
}

double avg(int x, int y, int z) {
    return (x+y+z)/3.0;
}

int main(int argc, const char * argv[]) {

    cout << avg(6, 7) << '\n';
    cout << avg(6, 7, 8) << '\n';
}
```

延伸閱讀：

<https://stackoverflow.com/questions/1657883/variable-number-of-arguments-in-c>

Pointer Argument

- 指向不同型態的指標是不同的

```
4  double avg(int *x, int y) {  
5      cout << "Avg1" << endl;  
6      return (*x+y)/2.0;  
7  }  
8  
9  double avg(double *x, int y) {  
10     cout << "Avg2" << endl;  
11     return (*x+y)/2.0;  
12 }  
13  
14 int main() {  
15     int i = 6;  
16     double d = 3.0;  
17     cout << avg(&i, 7) << "\n";  
18     cout << avg(&d, 7) << "\n";  
19 }
```

Reference

```
4 double avg(int& x, int& y) {
5     cout << "Avg1" << endl;
6     return (x+y)/2.0;
7 }
8
9 double avg(int x, int y) {
10     cout << "Avg2" << endl;
11     return (x+y)/2.0;
12 }
13
14 int main() {
15     int i = 6, j = 7;
16     cout << avg(i, j) << "\n";
17 }
```

int avg(int x, int y)
double avg(int x, int y)
→ 一樣不行!

```
lincyu@Chih-YudeMacBook-Pro Chapter6 % g++ 6_12_overload3.cpp
6_12_overload3.cpp:16:13: error: call to 'avg' is ambiguous
    cout << avg(i, j) << "\n";
                  ^~~
6_12_overload3.cpp:4:8: note: candidate function
double avg(int& x, int& y) {
    ^
6_12_overload3.cpp:9:8: note: candidate function
double avg(int x, int y) {
    ^
```

overloading accept: 1. 傳參形別不同
2. 傳參個數不同

Default Value

```
4  int avg(int x, int y) {
5      return (x+y)/2;
6  }
7
8  int avg(int x, int y, int z = 30) {
9      return (x+y)/3;
10 }
11
12 int main() {
13     cout << avg(6, 8) << endl;;
14 }
```

```
lincyu@Chih-YudeMacBook-Pro Chapter6 % g++ 6_12_overload4.cpp
6_12_overload4.cpp:13:13: error: call to 'avg' is ambiguous
    cout << avg(6, 8) << endl;;
               ^~~~
6_12_overload4.cpp:4:5: note: candidate function
int avg(int x, int y) {
   ^
6_12_overload4.cpp:8:5: note: candidate function
int avg(int x, int y, int z = 30) {
   ^
```

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

Recursion

- To understand recursion, you must understand recursion.



<http://programmers.stackexchange.com/questions/93826/how-do-i-explain-recursion-to-a-8-years-old-kid>

Recursion

- Factorial

- $n! = n * (n-1) * (n-2) * \dots * 2 * 1 = n * (n-1)!$

```
5  int factorial(int x) {
6      if (x == 1) {
7          return 1;
8      }
9      return x * factorial(x-1);
10 }
11
12 int main() {
13
14     cout << factorial(5) << "\n";
15     cout << factorial(10) << "\n";
16 }
```

Loop Version

- Factorial

```
5  int factorial(int x) {
6      int result = 1;
7      for (int i = 1; i <= x; i++) {
8          result = result * i;
9      }
10     return result;
11 }
12
13 int main() {
14
15     cout << factorial(5) << "\n";
16     cout << factorial(10) << "\n";
17 }
```


Fibonacci Number

- 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...
- $F_n = F_{n-1} + F_{n-2}$, with seed values $F_1 = 1, F_2 = 1$

Reference: https://en.wikipedia.org/wiki/Fibonacci_number

Practice

- (Practice 1) $1 + 2 + 3 + \dots + 10$
- (Practice 2) Fibonacci number (Recursion Version)
- (Practice 3) Fibonacci number (Loop Version)

Euclid's algorithm

- The greatest common divisor (gcd) of two positive integers is the largest integer that divides evenly into both of them.
 - For example, the greatest common divisor of 102 and 68 is 34 since both 102 and 68 are multiples of 34, but no integer larger than 34 divides evenly into 102 and 68.
- We can efficiently compute the gcd using the following property, which holds for positive integers p and q :
 - If $p > q$, the gcd of p and q is the same as the gcd of q and $p \% q$.

Euclid's algorithm

```
4  int gcd(int p, int q) {  
5      if (q == 0) return p;  
6      return gcd(q, p % q);  
7  }  
8  
9  int main() {  
10  
11      cout << gcd(24, 12) << "\n";  
12      cout << gcd(30, 25) << "\n";  
13      cout << gcd(102, 68) << "\n";  
14  }
```

Practice

- (Practice 1) Loop version of Euclid's algorithm.

Application

- Search a directory

MyDocument



School



Friend

hw1.docx

hw2.docx

School

school1.docx

school2.docx

Application

- Binary Search
 - Find the position of a target value within a sorted array
 - 23, 36, 78, 90, 102

Practice

- (Practice 1) Binary Search Loop Version
- (Practice 2) Binary Search Recursion Version

Nested Recursion

- What is the output of the below program?

```
4 void i(int x) {  
5     if (x == 0) {  
6         cout << "\n";  
7         return;  
8     }  
9     cout << "*";  
10    i(x-1);  
11 }
```

```
13 void o(int x) {  
14     if (x == 0) {  
15         return;  
16     }  
17     i(x);  
18     o(x-1);  
19 }
```

```
21 int main() {  
22     o(5);  
23 }
```

Practice

- (Practice 1) Print

1

12

123

1234

12345

APCS (USA)

Consider the following method.

```
public int multiply(int n)
{
    if (n == 0)
        return 1;
    else if (n % 2 == 1)
        return n;
    else
        return n * multiply(n - 2);
}
```

What will be returned by a call to `multiply(8)`?

- ☐ A 1
- ☐ B 8
- ☐ C (8)(7)(6)(5)(4)(3)(2)
- ☒ D (8)(6)(4)(2)
- ☐ E (7)(5)(3)

APCS (USA)

Suppose methods `f1` and `f2` are defined as follows:

```
public int f1(int x)
{
    if (x == 0)
        return 0;
    else
        return f2(x - 2);
}

public int f2(int x)
{
    if (x == 1)
        return 1;
    else
        return f1(x + 1) + x;
}
```

What value is returned as a result of the call `f1(5)`?

☐ A -4

☐ B 4

☒ C 6

APCS (Taiwan)

4. 右側函式兩個回傳式分別該如何撰寫，才能正確計算並回傳兩參數 **a**, **b** 之最大公因數 (Greatest Common Divisor) ?

(A) **a**, GCD(**b**, **r**)

(B) **b**, GCD(**b**, **r**)

(C) **a**, GCD(**a**, **r**)

(D) **b**, GCD(**a**, **r**)

```
int GCD (int a, int b) {  
    int r;  
  
    r = a % b;  
    if (r == 0)  
        return _____;  
    return _____;  
}
```

APCS (Taiwan)

7. 若以 **B(5,2)** 呼叫右側 **B()** 函式，總共會印出幾次 “base case”？

- (A) 1
- (B) 5
- (C) 10
- (D) 19

```
int B (int n, int k) {  
    if (k == 0 || k == n) {  
        printf ("base case\n");  
        return 1;  
    }  
    return B(n-1,k-1) + B(n-1,k);  
}
```

APCS (Taiwan)

10. 若以 **G(100)** 呼叫右側函式後，**n** 的值為何？

- (A) 25
- (B) 75
- (C) 150
- (D) 250**

```
int n = 0;

void K (int b) {
    n = n + 1;
    if (b % 4)
        K(b+1);
}

void G (int m) {
    for (int i=0; i<m; i=i+1) {
        K(i);
    }
}
```

APCS (Taiwan)

14. 若以 **F(15)** 呼叫右側 **F()** 函式，總共會印出幾行數字？

- (A) 16 行
- (B) 22 行
- (C) 11 行
- (D) 15 行**

```
void F (int n) {  
    printf ("%d\n" , n);  
    if ((n%2 == 1) && (n > 1)) {  
        return F(5*n+1);  
    }  
    else {  
        if (n%2 == 0)  
            return F(n/2);  
    }  
}
```


APCS (Taiwan)

20. 小藍寫了一段複雜的程式碼想考考你
是否了解函式的執行流程。請回答程式
最後輸出的數值為何？

- (A) 70
- (B) 80
- (C) 100
- (D) 190

```
int g1 = 30, g2 = 20;

int f1(int v) {
    int g1 = 10;
    return g1+v;
}

int f2(int v) {
    int c = g2;
    v = v+c+g1;
    g1 = 10;
    c = 40;
    return v;
}

int main() {
    g2 = 0;
    g2 = f1(g2);
    printf("%d", f2(f2(g2)));
    return 0;
}
```

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

標準函式庫

- 標準函式庫

- <https://zh.wikipedia.org/wiki/C%2B%2B%E6%A8%99%E6%BA%96%E5%87%BD%E5%BC%8F%E5%BA%AB>

- 標準樣板函式庫 (Standard Template Library, STL)

- <https://zh.wikipedia.org/wiki/%E6%A0%87%E5%87%86%E6%A8%A1%E6%9D%BF%E5%BA%93>

- 數學運算

- 時間函式

數學運算

- 次方、平方根、指數、對數

- `double exp (double x)` // 自然對數 e 的 x 次方
- `double log (double x)` // 自然對數 e 為基底的 x 對數值
- `double log10 (double x)` // 10 為基底的 x 對數值
- `double pow (double x, double y)` // x 的 y 次方
- `double sqrt (double x)` // x 的平方根

數學運算

```
1  #include <iostream>
2  #include <cmath>
3
4  using namespace std;
5
6  int main() {
7
8      cout << log10(1000) << "\n";
9      cout << sqrt((double)(3*3 + 4*4)) << "\n";
10 }
```

三角函數

```
1  #include <iostream>
2  #include <cmath>
3  #define PI 3.141592653589793
4
5  using namespace std;
6
7  int main() {
8
9      cout << "角度\tsin()\n";
10
11     for (int a = 30; a <= 180; a += 30) {
12         cout << a << "\t";
13         cout << sin(a * PI / 180) << "\n";
14     }
15 }
```

時間函式

```
1  #include <iostream>
2  #include <ctime>
3
4  using namespace std;
5
6  int main() {
7
8      clock_t starttime = clock();
9      for (int i = 0; i < 100000000; i++);
10     clock_t endtime = clock();
11
12     cout << CLOCKS_PER_SEC << "\n";
13     cout << ((double)(endtime-starttime) / CLOCKS_PER_SEC ) << "秒\n";
14 }
```

Practice

- (Practice 1) 比較迴圈與遞迴的執行效率 (未經編譯器最佳化)

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- **Function Template**
- `std::optional<>` (C++17)

Function Template

会考

Function Overloading 不太一樣!

Function Template

有範形, more powerful!

```
1  #include<iostream>
2  using namespace std;
3
4  template<typename T> T average(T a, T b);
5
6  int main() {
7      cout << average(6, 9) << endl;
8      cout << average(3.6, 5.8) << endl;
9  }
10
11 template<typename T> T average(T a, T b) {
12     return (a+b)/2;
13 }
```

一个資料形態 (but 範形)

Outline

- Introduction to functions
- Variable Scope
- Inline functions
- Macro
- Overloading
- Recursive functions
- C++ Standard library
- Function Template
- `std::optional<>` (C++17)

std::optional<> (C++17)

沒教、不考

```
1  #include<iostream>
2  #include<optional>
3
4  using namespace std;
5
6  optional<int> f(int x) {
7      if (x >= 0) {
8          return x;
9      } else {
10         return std::nullopt;
11     }
12 }
```

```
14  int main() {
15      auto result1 { f(3) };
16      auto result2 { f(-2) };
17
18      if (result1 != std::nullopt) {
19          cout << "Result 1\n";
20      }
21      if (result2 != std::nullopt) {
22          cout << "Result 2\n";
23      }
24 }
```